

SnapForge

Audio multiroom per Raspberry Pi

Costruito in 5 giorni

Claudio | Gennaio 2026

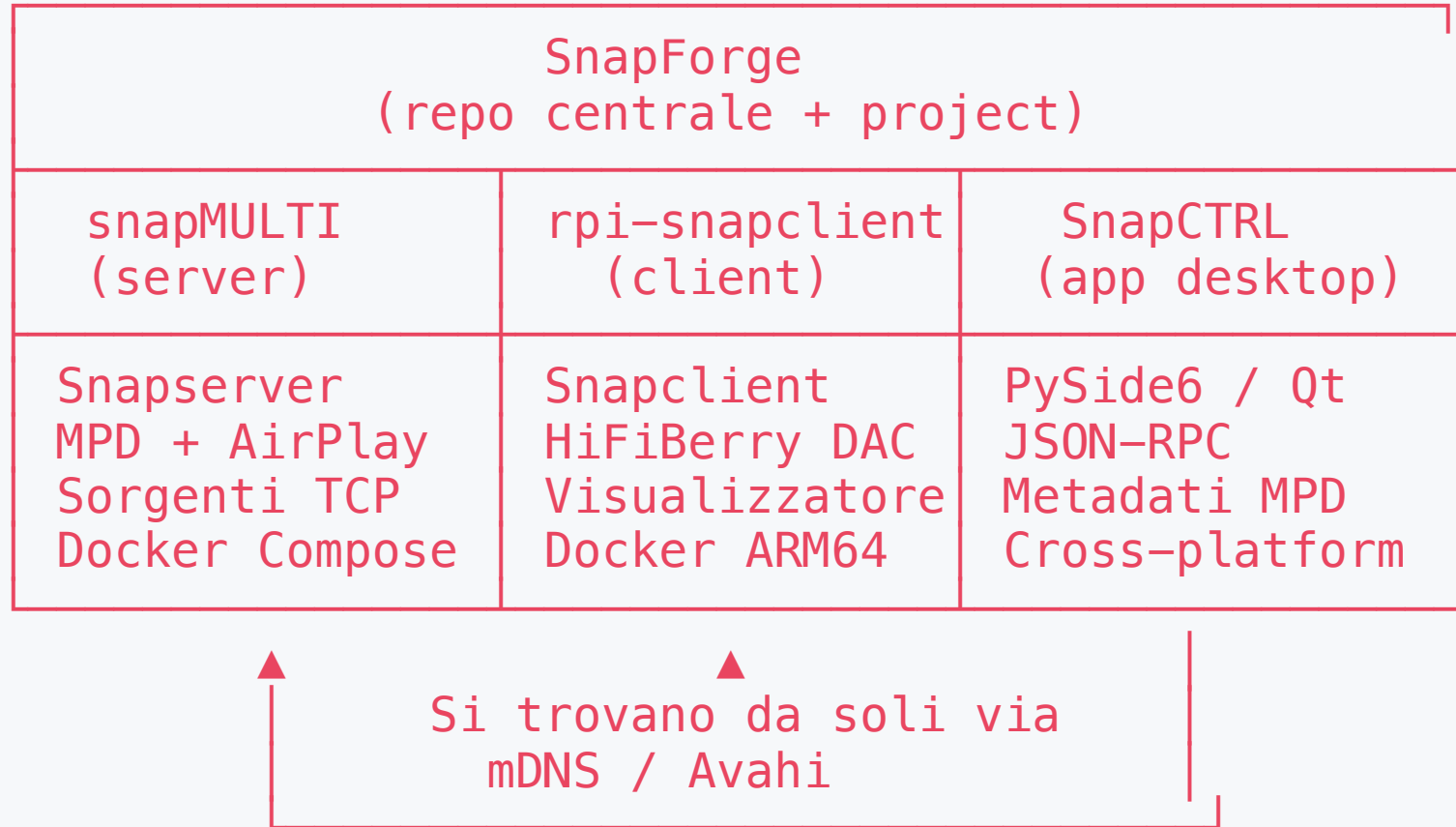
Il problema

Far funzionare l'audio multiroom su Raspberry Pi non e' semplice:

- **Snapcast** e' potente, ma non ha un setup "chiavi in mano"
- Configurare DAC, discovery di rete e metadati e' tutto manuale
- Non esiste un'app desktop unificata per controllare tutto
- Ogni pezzo vive per conto suo, senza una visione d'insieme

Obiettivo: da Raspberry Pi "nudi" a un sistema multiroom funzionante
con un solo `docker compose up`

Come e' fatto



Tre componenti indipendenti che si scoprono automaticamente in rete.

snapMULTI — Il Server

Il cuore: gestisce le sorgenti audio e distribuisce ai client

- **3 sorgenti audio:** MPD (musica), TCP (ingresso linea), AirPlay (iPhone/Mac)
- I client lo trovano da soli in rete (mDNS)
- Tutta la configurazione in file nella cartella `config/`
- Build e deploy automatici con GitHub Actions

```
# docker-compose.yml — basta questo per partire
services:
  snapserver:
    image: snapmulti-server
    network_mode: host
    volumes:
      - /run/dbus:/run/dbus          # Per la discovery
      - ./config:/etc/snapserver    # Configurazione
```

rpi-snapclient — Il Client

Trasforma un Raspberry Pi in un player audio con display

Cosa fa	Dettaglio
Schede audio	11 supportate (HiFiBerry, IQaudio, JustBoom...)
Display	Copertina album sullo schermo collegato
Visualizzatore	Barre audio in tempo reale (CAVA)
Discovery	Trova il server da solo, zero configurazione
Metadati	Titolo, artista, album dalla sorgente MPD
Build	Docker nativo ARM64, CI su runner dedicato

v1.0.0 rilasciata il 26 Gennaio — 2 giorni dopo il primo commit

SnapCTRL — L'App Desktop

Controlla tutto il sistema audio dal tuo computer

- Vedi tutti i client connessi in tempo reale
- Volume e mute per ogni stanza / gruppo
- Cambia sorgente audio al volo
- **"In riproduzione"**: titolo, artista e copertina album
- Latenza di rete per ogni client
- Funziona su macOS, Linux e Windows

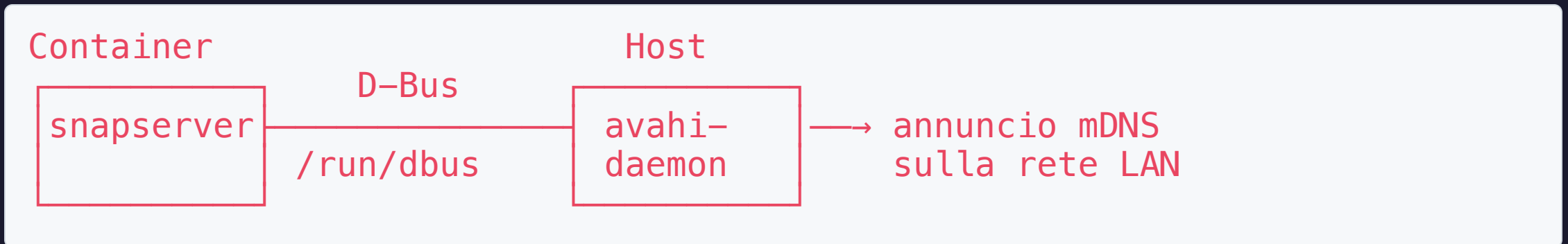


La sfida: discovery in Docker

I container Docker non vedono il broadcast mDNS della rete locale.

I client devono trovare il server senza configurare IP a mano.

Soluzione: il container usa Avahi dell'host tramite D-Bus



- Il server si annuncia come `_snapcast._tcp`
- I client lo trovano automaticamente
- Nessun IP da configurare

CI/CD — Build e deploy automatici

Un Raspberry Pi fa da runner GitHub Actions

Push su main

- Controlla sintassi e configurazione
- Build Docker (ARM64 nativo, no emulazione)
- Test (se il server e' raggiungibile)
- Deploy sui Raspberry Pi di destinazione

Scelta	Perche'
Runner su RPi	Build ARM64 nativi, veloce, senza emulazione
Immagini locali	Non serve un registry per uso domestico
Deploy via SSH	Semplice, affidabile, tracciabile
Hook pre-push	Cattura errori prima della CI

Gestione del progetto

Un unico GitHub Project per tutti i repository

Cosa	Dettaglio
Campi custom	Componente, Priorita', Tipo, Effort
Label	17 etichette uguali su tutti e 4 i repo
Template issue	Bug report + Feature request con form guidati
Tracciamento	Vista per componente e cross-ecosistema

Tutti i repo parlano la stessa lingua:
stesse etichette, stessi template, stessa scala di priorita'.

Timeline — Giorno per giorno

Giorno	Cosa e' successo
24 Gen	Primo commit rpi-snapclient: player + display copertine
	Primo commit SnapCTRL: app desktop di controllo
25 Gen	Client: supporto 11 schede audio, CI/CD, Docker multi-arch
26 Gen	Client: release v1.0.0
	Server: nasce snapMULTI, discovery mDNS funzionante
27 Gen	Server: 3 sorgenti (MPD + TCP + AirPlay)
	Client: visualizzatore audio, autodiscovery, metadati
	SnapCTRL: pannello sorgenti, sync volume, bundle macOS
28 Gen	SnapCTRL: metadati MPD + copertine album
	Nasce SnapForge: meta-repo + project board unificato
	Tutti i repo: label, template, CI su runner dedicato

Stack tecnologico

Livello	Tecnologia
Motore audio	Snapcast (server + client)
Sorgenti	MPD, AirPlay (shairport-sync), TCP pipe
Container	Docker + Compose, ARM64 nativo
App desktop	PySide6 / Qt 6
API	Snapcast JSON-RPC, protocollo MPD
Discovery	Avahi / mDNS (<code>_snapcast._tcp</code>)
Visualizzazione	CAVA (Console Audio Visualizer)
CI/CD	GitHub Actions, runner self-hosted ARM64
Gestione progetto	GitHub Projects v2

Scelte di design

Scelta	Motivazione
Docker ovunque	Riproducibile, facile da aggiornare e ripristinare
Rete host	Necessario per mDNS, accettabile su rete domestica
Runner self-hosted	Build nativi ARM64, niente emulazione lenta
PySide6 invece di Electron	Prestazioni native, app piu' leggera
MPD come sorgente primaria	Solido, metadati ricchi, riproduzione senza pause
File di config (no env vars)	Piu' facili da versionare, leggere e condividere
Un solo project board	Vista unica su tutti i componenti

Prossimi passi

- **Viste progetto:** Kanban, per componente e Roadmap
- **Automazioni:** triage automatico, stato "Done" al merge delle PR
- **CLI SnapForge:** installer unificato per tutto l'ecosistema
- **Aggiornamenti OTA:** pull automatico delle nuove immagini Docker
- **Web UI:** alternativa leggera a SnapCTRL per tablet e telefono
- **Multi-server:** Snapcast federato tra piu' sedi

Grazie

SnapForge Ecosystem

github.com/lollonet

Repo	Ruolo
<code>snapforge</code>	Repo centrale + project board
<code>snapMULTI</code>	Server Snapcast multi-sorgente
<code>rpi-snapclient-usb</code>	Client RPi con supporto DAC
<code>snapctrl</code>	Controller desktop PySide6

Da zero ad audio multiroom in 5 giorni.